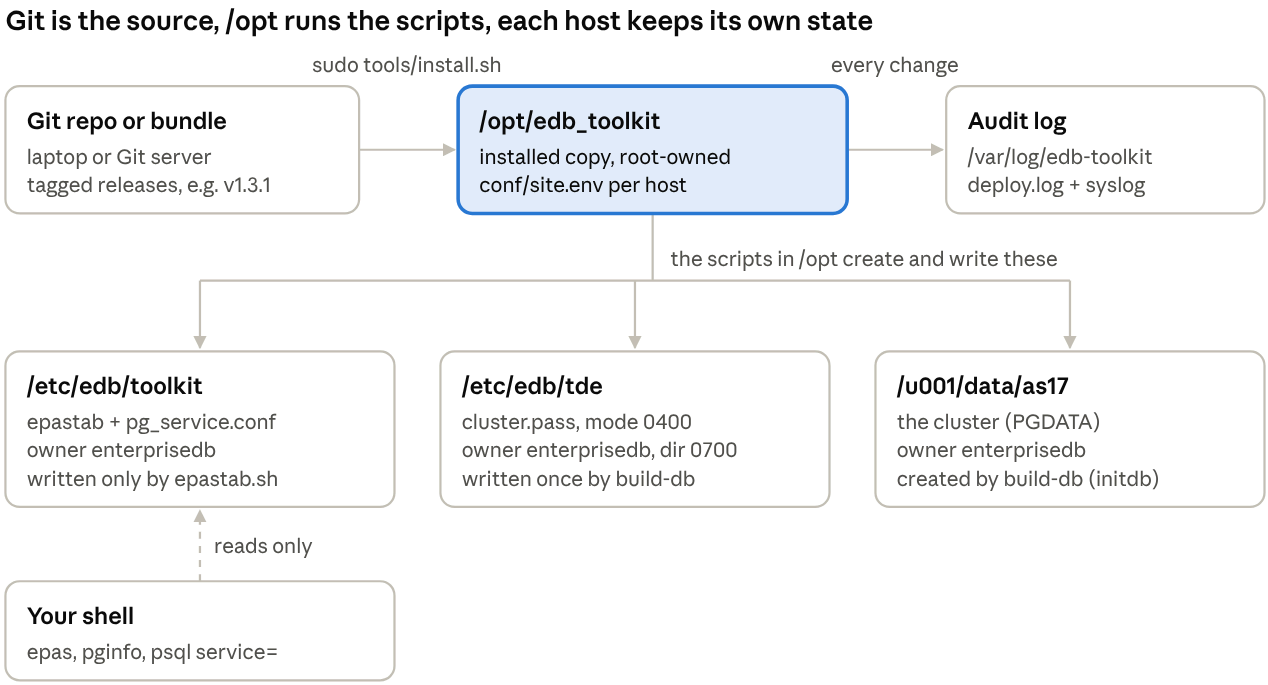
edb_toolkit — Beginner's Operating Guide

Nov 4, 2026 · @Vinay

0. How the toolkit fits together

The toolkit lives in five places on a server, and each place has exactly one owner and one writer. Once you know which is which, every command in this guide makes sense. Written against `edb_toolkit` v1.3.1 (commit 7287054).

Place	Path	Owner	What it holds	Who changes it
Source copy	Git checkout or unpacked bundle (e.g. `~/edb_toolkit`)	you	what you are about to install	`git pull` / a new bundle
Installed toolkit	`/opt/edb_toolkit`	root, read-only for everyone else	the scripts that actually run, plus this host's `conf/site.env`	only `tools/install.sh` (root edits `site.env`)
Cluster registry	`/etc/edb/toolkit/` (`epastab`, `pg_service.conf`)	enterprisedb	which clusters SHOULD exist on this host	only `tools/epastab.sh` (you may hand-write `pg_service.local.conf`)
TDE passphrase files	`/etc/edb/tde/<cluster>.pass`	enterprisedb, dir 0700, file 0400	the passphrase that unlocks each encrypted cluster	`epas-base-build-db.sh`, once; a copy lives in your vault
The cluster	e.g. `/u001/data/as17`	enterprisedb	the database itself	EPAS



where the toolkit lives on a server · 7 parts

Only `install.sh` changes `/opt`; only the toolkit's scripts write under `/etc/edb`; your shell reads the registry and never writes it.

Every change a script makes is also written as an `[AUDIT]` line to `/var/log/edb-toolkit/deploy.log` and to syslog (tag `edb-toolkit-audit`).

Who runs what. Root does packages, directories, systemd and installing the toolkit. The database OS user (`enterprisedb`, called `SYSTEM_USER` in the config) does everything else: building the cluster, the registry, daily work. If a step below starts with `sudo`, it is a root step; if it starts after `sudo -iu enterprisedb`, it is a DB-user step.

Conventions you will see everywhere

- Cluster names are `as<major>`, plus `_<suffix>` for a second cluster of the same version: `as17`, `as17_rpt`.
- Environment tags are `DEV`, `SIT`, `PAT`, `PROD`. A DR copy of production is `PROD`. The toolkit never guesses a tag; you always give it.
- Scripts that change something first print the values they will use, then ask `About to: … — Proceed? (y/N)`. `--yes` skips the question. In a non-interactive run (Ansible, cron) they proceed without asking.
- `--env <file>` picks a config file; without it, scripts use `conf/site.env` inside the toolkit. `--help` works on every script.

- Log lines look like `[2026-11-04 22:14:07.512] [INFO] [epastab.sh] Wrote …`. In the examples below the timestamp is trimmed.

Nothing has been run on a real EPAS server yet (that is the next roadmap item), so the example output below is what the scripts print according to their code; the real thing may differ in small details such as package lists and timings.

1. Get the toolkit onto a server

Pick a tag (e.g. `v1.3.1`), get that exact version onto the server by Git or by bundle, then run `sudo tools/install.sh` from it. Both roads end in the same `/opt/edb_toolkit`.

Road A — the server can reach Git

```
# on the server, as your own admin account (not enterprisedb)
cd ~ && git clone https://<your-git-server>/dba/edb_toolkit.git
cd edb_toolkit
git checkout v1.3.1        # install a tag, never "whatever main is today"
make test                  # 51 unit tests; needs no database and no root
sudo tools/install.sh      # same as: sudo make install
```

Road B — the server cannot reach Git

On your laptop, build a bundle from the tag. It contains only committed files (never your local `conf/site.env`) and warns you if you have uncommitted changes.

```
cd edb_toolkit && git checkout v1.3.1
make bundle
# Bundle:   …/edb_toolkit/dist/edb_toolkit-v1.3.1.tar.gz
# Checksum: …/edb_toolkit/dist/edb_toolkit-v1.3.1.tar.gz.sha256
# Version:  v1.3.1 (<commit date>)
scp dist/edb_toolkit-v1.3.1.tar.gz* admin@dbhost01:/tmp/
```

On the server, check the file arrived intact before you unpack it:

```
cd /tmp
sha256sum -c edb_toolkit-v1.3.1.tar.gz.sha256
# edb_toolkit-v1.3.1.tar.gz: OK
tar xzf edb_toolkit-v1.3.1.tar.gz
sudo edb_toolkit/tools/install.sh
```

What install.sh does and prints

It copies the toolkit to `/opt/edb_toolkit`, owned by root (dirs 755, files 644, `*.sh` 755), so enterprisedb can run the tools but cannot change scripts that root also runs. It keeps the previous install as `/opt/edb_toolkit.prev`, writes a `VERSION` file and links `/usr/local/bin/epasenv`.

```
  SRC     = /tmp/edb_toolkit
  PREFIX  = /opt/edb_toolkit
  VERSION = v1.3.1 (2026-11-04)
  CURRENT = none
About to: install edb_toolkit v1.3.1 (2026-11-04) into /opt/edb_toolkit
(current: none; it is kept as /opt/edb_toolkit.prev) — Proceed? (y/N): y
[INFO] [install.sh] Linked /usr/local/bin/epasenv ->
/opt/edb_toolkit/tools/epasenv.sh  (use: . epasenv <cluster>)
[AUDIT] [install.sh] edb_toolkit v1.3.1 (2026-11-04) installed in
/opt/edb_toolkit (previous: none)
[INFO] [install.sh] Done. Roll back with: sudo mv /opt/edb_toolkit
/opt/edb_toolkit.bad && sudo mv /opt/edb_toolkit.prev /opt/edb_toolkit
```

Check it:

```
cat /opt/edb_toolkit/VERSION
# v1.3.1 (2026-11-04)
# installed 2026-11-04 22:10:31 EST by vinay from /tmp/edb_toolkit
ls -l /usr/local/bin/epasenv
```

Give the server its config: conf/site.env

`install.sh` never copies or overwrites `conf/*.env`, so each server keeps its own. Create it once, as root:

- **New, empty server:** copy the template and edit it (step 2 lists the values that matter).
- **Server that already runs EPAS:** let the generator detect version, data directory, port, OS user and network, confirm each value, and write the file.

```
# new server
sudo cp /opt/edb_toolkit/conf/site.env.example /opt/edb_toolkit/conf/site.env
sudo vi /opt/edb_toolkit/conf/site.env

# existing server (asks you to confirm every value; --instance picks one of
several clusters)
sudo /opt/edb_toolkit/modules/01-provision/generate-site-env.sh

# either way: enterprisedb must be able to READ it (it holds no secrets)
sudo chmod 644 /opt/edb_toolkit/conf/site.env
```

> **Pro tips**
>
> - Before you touch an unfamiliar server, take a read-only look: `sudo /opt/edb_toolkit/tools/get-edb-inventory.sh` lists EDB packages, running instances, EFM and pgBackRest. It changes nothing.
> - Never edit scripts under `/opt/edb_toolkit`. Fix them in Git, tag, and reinstall; otherwise the next install silently throws your fix away.
> - Keep a copy of each server's `site.env` in a private repo or your config management. It is the one file on the server that is not rebuilt from Git.

2. Build a new cluster

A new cluster is two runs: part A as root prepares the host, part B as enterprisedb creates the encrypted cluster and registers it. Both read the same `conf/site.env`, and both are safe to re-run: every step checks what is already there.

2.1 Fill in site.env

These are the values the build uses. The rest of the file (replication, EFM, upgrade) is for other modules.

Setting	Example	What it means
`EPAS_VERSION`	`17`	major version to install
`SERVICE_NAME`	`edb-as-17`	the systemd unit from the EDB package

Setting	Example	What it means
`SYSTEM_USER` / `SYSTEM_GROUP`	`enterprisedb`	DB OS account; created by the EDB package
`SRC_BINARY_DIR`	`/usr/edb/as17`	where the package puts binaries; becomes a symlink
`BINARY_TOP`	`/u001/binary/as17`	where the binaries really live afterwards
`DATA_TOP`	`/u001/data/as17`	the cluster's data directory (PGDATA)
`PRIMARY_PORT`	`5444`	port, written to `conf.d/01-network.conf`
`CLUSTER_NAME`	`as17`	registry name; `as17_rpt` for a second v17 cluster
`ENV_TAG`	`DEV`	`DEV`, `SIT`, `PAT` or `PROD`; empty = you must pass `--tag`
`TDE_PASS_DIR` / `TDE_PASS_FILE`	`/etc/edb/tde` / `/etc/edb/tde/as17.pass`	where the TDE passphrase file goes
`TDE_WRAP_CMD` / `TDE_UNWRAP_CMD`	`openssl enc … -pass file:${TDE_PASS_FILE} …`	leave as in the template

ENV_TAG. Set it in `site.env`, or pass `--tag` on the command line; `--tag` wins. If both are empty, part B stops before changing anything. Getting the tag right matters: it drives the prompt colour every DBA will see (step 4).

The TDE passphrase. Never put it in `site.env`. Part B takes it from the first of these that exists:

1. the file `TDE_PASS_FILE` already exists (written by a vault agent or Ansible) — used as is;
2. the variable `$TDE_PASSPHRASE` — written to the file;
3. you are at a terminal — it asks twice, hidden, and writes the file.

It must be at least 12 characters and one line. The file ends up mode 0400 in a 0700 directory, and `postgresql.conf` stores only the file's path. **Put the passphrase in your vault before you start the cluster**: without it the cluster cannot start and no backup can be restored.

2.2 Part A, as root

```
sudo /opt/edb_toolkit/modules/01-provision/epas-base-build-root.sh
```

It installs the EPAS packages, moves the binaries to `BINARY_TOP` (leaving `/usr/edb/as17` as a symlink), creates `DATA_TOP`, writes a systemd drop-in, runs `epastab.sh init`, and creates `/etc/edb/tde`. Expected tail of the output:

```
[INFO]  Creating compatibility symlink: /usr/edb/as17 -> /u001/binary/as17
[INFO]  Configuring directory ownership to enterprisedb:enterprisedb
[INFO]  Creating Systemd Unit Overrides in /etc/systemd/system/edb-as-
17.service.d/override.conf
[INFO]  Preparing the cluster registry directory (epastab.sh init)
[AUDIT] epastab: registry directory /etc/edb/toolkit ready, owner
enterprisedb:enterprisedb (mode 755)
[INFO]  Created /etc/edb/tde (owner enterprisedb:enterprisedb, mode 700)
[AUDIT] Day 1 part A (root) done: EPAS 17, binaries /u001/binary/as17, data
dir /u001/data/as17, service edb-as-17, TDE dir /etc/edb/tde
[INFO]  Next, as enterprisedb (no root needed):
[INFO]    sudo -iu enterprisedb /opt/edb_toolkit/modules/01-provision/epas-
base-build-db.sh --tag <DEV|SIT|PAT|PROD>
```

Watch for `Warning: The following packages could not be installed`. Part B's `conf.d/04-extensions.conf` preloads `edb_wait_states`, `edb_stat_monitor`, `query_advisor` and `sql-profiler`; if one of those packages is missing, the server will refuse to start.

2.3 Part B, as enterprisedb

```
sudo -iu enterprisedb /opt/edb_toolkit/modules/01-provision/epas-base-build-
db.sh --tag DEV
```

All checks run before anything changes, so a failed check leaves the host exactly as it was. Then it asks for the passphrase and builds:

```
About to: create TDE cluster as17 in /u001/data/as17 (initdb --data-
encryption=256), stage conf.d templates, register it as DEV — Proceed? (y/N):
y
TDE passphrase for as17 (min 12 characters, hidden):
Same passphrase again:
[INFO]  Wrote /etc/edb/tde/as17.pass (mode 0400). Keep the passphrase in your
vault too: …
[INFO]  TDE wrap/unwrap round trip OK
[INFO]  Initializing Database Cluster with Native TDE Engine
        … initdb output …
[INFO]  Unwrap command stored in postgresql.conf:
data_encryption_key_unwrap_command = 'openssl enc -d -aes-128-cbc -pbkdf2 -
pass file:/etc/edb/tde/as17.pass -in %p'
[INFO]  Staged conf.d/01-network.conf   (… 02-memory, 03-logging, 04-
extensions, 05-compatibility, 99-local)
[AUDIT] epastab: registered as17:17:/u001/data/as17:5444:edb-as-17:DEV in
/etc/edb/toolkit/epastab
[INFO]  Wrote /etc/edb/toolkit/pg_service.conf (1 cluster(s))
[INFO]  Next: start the cluster
[INFO]    sudo systemctl start edb-as-17    (allowed for enterprisedb by
conf/sudoers.d/epas-automation)
[INFO]    then:  . epasenv as17   and   psql service=as17
```

`conf.d/02-memory.conf` is sized from the host's RAM at build time (shared_buffers 25%, effective_cache_size 75%). Your own overrides go in `conf.d/99-local.conf`; re-runs never overwrite any `conf.d` file.

2.4 Start it and prove it works

The `sudo systemctl start` line works for enterprisedb only after root has installed the sudoers file once (read section 7 first: there is a mismatch with `pgstart`):

```
sudo cp /opt/edb_toolkit/conf/sudoers.d/epas-automation /etc/sudoers.d/
sudo visudo -c -f /etc/sudoers.d/epas-automation
```

Then, as enterprisedb:

```
sudo systemctl start edb-as-17
. epasenv as17
psql service=as17 -c "select version();"
/opt/edb_toolkit/tools/epastab.sh check     # expect: RESULT registry matches
this host
```

The old one-shot command `sudo epas-base-build.sh --tag DEV` still runs A then B for you, but it is a wrapper that will be removed. Use the two scripts.

> **Pro tips**
>
> - Prove the cluster can unlock itself without you: `sudo systemctl restart edb-as-17`. If the restart works, the unwrap command and pass file are right.
> - Check what got stored: `grep unwrap /u001/data/as17/postgresql.conf` must show `-pass file:/etc/edb/tde/as17.pass`, never the passphrase itself.
> - Back up the cluster with pgBackRest before handing it over; a backup you have never restored is a hope, not a backup.

3. Bring an existing server under the toolkit

An existing server needs no rebuild: root creates the registry directory once, enterprisedb imports what is already running, and `check` confirms the registry matches reality. Nothing here touches a cluster.

Install the toolkit first (step 1). Then:

3.1 Look before you touch (read-only)

```
sudo /opt/edb_toolkit/tools/get-edb-inventory.sh
```

Note each cluster's version, data directory, port, systemd unit and whether it is production. You need these to judge the import plan in 3.3.

3.2 Root, once per host: init

```
sudo /opt/edb_toolkit/tools/epastab.sh init --owner enterprisedb
# or take the owner from SYSTEM_USER in site.env:
sudo /opt/edb_toolkit/tools/epastab.sh init --env
/opt/edb_toolkit/conf/site.env
```

```
About to: create /etc/edb/toolkit owned by enterprisedb:enterprisedb, mode
755 - the database user then maintains /etc/edb/toolkit/epastab without root
— Proceed? (y/N): y
[AUDIT] [epastab.sh] epastab: registry directory /etc/edb/toolkit ready,
owner enterprisedb:enterprisedb (mode 755)
```

Only `/etc/edb/toolkit` is handed to enterprisedb. `/etc/edb` itself stays root's, because EDB packages (EFM, for example) keep files there. Running `init` again is harmless: it says `already set up … nothing to do`.

3.3 As enterprisedb: import, dry run first

`import` finds running clusters plus any installed `edb-as-*` units (so stopped ones too), and registers them all with the ONE tag you give.

```
sudo -iu enterprisedb
alias epastab=/opt/edb_toolkit/tools/epastab.sh     # saves typing; put it at
the end of ~/.bashrc
epastab import --tag PROD --dry-run
```

```
Clusters to register in /etc/edb/toolkit/epastab:
  as15:15:/u001/data/as15:5445:edb-as-15:PROD
  as17:17:/u001/data/as17:5444:edb-as-17:PROD
(dry run - nothing written)
```

Read each line as `name:version:data_directory:port:service:tag`. If the plan is right, run it for real:

```
epastab import --tag PROD
# About to: register 2 cluster(s) in /etc/edb/toolkit/epastab with tag PROD —
Proceed? (y/N): y
# [AUDIT] epastab: imported 2 cluster(s) into /etc/edb/toolkit/epastab: …
# [INFO]  Wrote /etc/edb/toolkit/pg_service.conf (2 cluster(s))
```

When import cannot decide, it skips the cluster with a warning instead of guessing. Register those by hand with `add`:

- **A second cluster of the same version** (name `as17` already taken): give it a suffix.
- **A stopped cluster with no `edb-as-*` unit**: import cannot see it.
- **A host with mixed tags** (rare): import uses one tag per run, so `add` the odd one.

```
epastab add --name as17_rpt --data /u001/data/as17_rpt --port 5446 --tag PROD
--service -
#   --service -  = started with pg_ctl, not systemd; leave --service out to
let it detect edb-as-17
epastab list
```

```
NAME      VERSION  PORT  TAG   STATE    SERVICE    DATA DIRECTORY
as15      15       5445  PROD  running  edb-as-15  /u001/data/as15
as17      17       5444  PROD  running  edb-as-17  /u001/data/as17
as17_rpt  17       5446  PROD  stopped  -          /u001/data/as17_rpt
```

3.4 check

```
epastab check; echo "exit code $?"
```

```
INFO  as15: PROD, running
INFO  as17: PROD, running
INFO  as17_rpt: PROD, stopped
RESULT registry matches this host
exit code 0
```

Exit code 0 = clean, 2 = drift found, 1 = error. Section 5 explains each drift message. Run `check` as enterprisedb or root; any other user gets `NOTE … not readable` lines, because PGDATA is mode 0700.

Finally, if you plan to run other modules on this host, create `site.env` with `generate-site-env.sh` (step 1), and install the DBA `.bashrc` (step 4).

> **Pro tips**
>
> - Always `--dry-run` first on production. The registry drives the prompt colour, so a wrong tag on a PROD box is a real risk, not a cosmetic one.
> - The registry stores what should exist, not who is primary. Primary/standby changes on failover, so the toolkit always reads it live (`pgrepl` , `pghealth`).
> - `epastab remove --name as15` only removes the registry line. It never touches the cluster.

4. Daily use as the DB user

Day to day you log in as enterprisedb, point your shell at one cluster with `epas <name>`, and use the `pg*` helpers and `psql service=<name>`. The prompt always tells you which cluster, and which environment, you are about to type into.

4.1 Install the DBA .bashrc (once per server)

`install.sh` does not touch anyone's home directory, so copy it yourself:

```
sudo -iu enterprisedb
cp -p ~/.bashrc ~/.bashrc.orig.$(date +%F)      # keep the old one
cp /opt/edb_toolkit/tools/pg_bashrc.txt ~/.bashrc
exit
sudo -iu enterprisedb                            # log in again to load it
```

At login it picks a default cluster (the only running one, else `as17`, else the highest registered), sets the environment and prints `pginfo`. It sets `PGUSER` to the OS account and `PGDATABASE=edb`; anything you export yourself wins.

4.2 Switch clusters: epas and . epasenv

`epas` is a shortcut for `. epasenv`. Both change the current shell: `PATH`, `PGHOME`, `PGDATA`, `PGPORT`, `PGHOST` (socket directory), plus `EPAS_CLUSTER` and `EPAS_TAG` for the prompt.

```
epas as17_rpt                    # by registry name
epas 15                          # by version -> as15
. epasenv /u001/data/as17        # by data directory
. epasenv                        # numbered menu
. epasenv -l                     # list only, change nothing
```

```
EPAS 17 (17.x) - as17_rpt [PROD] - /u001/data/as17_rpt - port 5446 (<source>)
- running (pg_ctl)
```

The bracket after the port says where the port came from. When it ends in `verify!`, the port was guessed (port map or EPAS default), so check it before you trust it. Why the dot in `. epasenv`? A script you *run* is a child process and cannot change your shell; a script you *source* (`.`) can. Running `epasenv` without the dot just prints that reminder.

4.3 See where you are: pginfo

```
======================================
  EPAS Environment
======================================
Active Version : 17 (17.x)
Cluster        : as17 [PROD]
View           : full (owner)
Binaries       : /u001/binary/as17
Data Directory : /u001/data/as17
Port           : 5444 (<source>)
Instance       : running (PID 41207)
systemd Unit   : edb-as-17.service: active / enabled
Started By     : systemd (edb-as-17.service)
Control Mode   : systemd for stop/restart (setting: auto)
Client         : user=enterprisedb (OS user) db=edb
psql=/u001/binary/as17/bin/psql
Detection      : library 1.0.0 (/opt/edb_toolkit)
Services       : /etc/edb/toolkit/pg_service.conf (psql service=<name>)
--------------------------------------
  Host  (hostname, IP/subnet, OS, CPU, memory, last reboot)
```

`View : limited` means you are not the cluster owner, so some values are fallbacks. `Detection : built-in` means the toolkit library was not found and the `.bashrc` used its own copy.

4.4 Start, stop, reload: pgstart, pgstop and friends

Command	Does
`pgstatus`	pg_ctl status + systemd unit state
`pgstart`	start; does nothing if already running
`pgstop [smart|fast|immediate]`	stop; `fast` is the default with pg_ctl
`pgrestart`	restart
`pgreload`	re-read config (SIGHUP); no downtime
`pgmode auto|systemd|pg_ctl`	default method for this shell

In `auto` mode (the default) the helpers follow whoever controls the cluster right now: started by systemd → `systemctl` via sudo; started by `pg_ctl` → `pg_ctl`; stopped → systemd if the unit exists. They refuse unsafe combinations, for example `systemctl stop` on a cluster that `pg_ctl` started (systemd would not stop it). To hand a manually started cluster back to systemd: `pgstop --pgctl; pgstart --systemd`.

```
$ pgstop
+ systemctl stop edb-as-17.service
edb-as-17.service: inactive
```

4.5 Connect by name: psql service=

`epastab.sh` generates `/etc/edb/toolkit/pg_service.conf` from the registry: one entry per cluster, over the local socket, `dbname=edb`. The `.bashrc` and `epasenv` set `PGSYSCONFDIR=/etc/edb/toolkit` so libpq finds it.

```
psql service=as17_rpt                            # no host/port to remember
psql "service=as17 dbname=appdb"                 # override the database
PGSERVICE=as15 pg_dump -Fc -f /backup/app.dump appdb
cat /etc/edb/toolkit/pg_service.conf             # see every entry
```

Never edit `pg_service.conf`; it is rewritten on every `add`, `remove` and `import`. Your own entries go in `/etc/edb/toolkit/pg_service.local.conf`, which is appended unchanged. `epasenv` clears an old `PGSERVICE` (with a notice), because it would override host and port.

4.6 What the prompt colours mean

Prompt	Colour	Meaning
`[PROD as17:5444][enterprisedb@db01 ~]$`	bold red	production (or DR of production): slow down
`[PAT as17:5444][…]$`	amber	pre-production / acceptance
`[SIT as17:5444][…]$` / `[DEV …]`	green	test and development
`[enterprisedb@db01 ~] [EPAS:17:5444]$`	none	cluster not in the registry: register it (step 3)

The tag is always printed as text; colour is extra. `NO_COLOR=1` removes the colours, `EDB_PROMPT=no` keeps your own prompt.

4.7 Other helpers worth knowing

Command	Shows
`pghealth`	version, standby yes/no, uptime, connections, database sizes and XID age, transactions open > 5 min, replication slots
`pgactivity` / `pglocks`	what is running now / who blocks whom
`pgrepl`	replication state and lag
`pglog` / `pgjournal`	server log / systemd journal
`pgwal` / `pgsize`	WAL directory / data directory size
`cdpg` / `cdlog` / `pgconf` / `pghba`	jump to PGDATA, the log dir, or edit the config files
`epaslist`	every installed version with its unit, port and data dir

> **Pro tips**
>
> - Before anything destructive (`DROP` , `pgstop immediate` , `pg_resetwal`), read the prompt tag out loud. It sounds silly; it has saved many production databases.
> - After editing a `conf.d` file, `pgreload` , then `psql -c "select name, setting, error from pg_file_settings where error is not null"` shows typos, and `select name from pg_settings where pending_restart` shows what needs a restart.
> - `ALTER SYSTEM` writes to `postgresql.auto.conf` , which beats every `conf.d` file. Keep settings in `conf.d` so they are visible and reviewable.

5. Which tool when, and what to do about drift

One rule covers most choices: `epastab.sh` is the only thing that writes the registry, `epasenv` / `epas` only read it, and the build scripts call `epastab.sh` for you.

5.1 Which tool when

You want to…	Use	As
see what is on an unfamiliar host	`tools/get-edb-inventory.sh`	root (fullest view)
install or update the toolkit	`tools/install.sh`	root
create `site.env` on a host that already runs EPAS	`modules/01-provision/generate-site-env.sh`	root
build a brand-new cluster	`epas-base-build-root.sh`, then `epas-base-build-db.sh`	root, then enterprisedb
set up the registry on a host	`epastab.sh init`	root, once
register what already runs	`epastab.sh import --tag …`	enterprisedb
register one cluster by hand	`epastab.sh add …`	enterprisedb
unregister a cluster (keeps the cluster)	`epastab.sh remove --name …`	enterprisedb
list registered clusters with state	`epastab.sh list` or `. epasenv -l`	anyone
compare the registry with reality	`epastab.sh check`	enterprisedb or root
rebuild `pg_service.conf` after a socket change	`epastab.sh services`	enterprisedb
point your shell at a cluster	`epas <name>` / `. epasenv <name>`	anyone
start, stop, reload	`pgstart`, `pgstop`, `pgreload`	enterprisedb
connect	`psql service=<name>`	anyone with DB access
SSL, patching, upgrade, EFM, backup, tuning, Thales TDE	`modules/02-ssl` … `modules/08-tde` (each has a README)	per README

5.2 When check reports drift

The registry is the *declared* state; the host is the *observed* state. Drift means they disagree, and your first job is to decide **which side is wrong**. If the change on the host was intended, update the registry. If it was not, fix the host. Never hand-edit `epastab`: `check` flags malformed lines, and every write tool refuses to run until they are fixed.

`epastab.sh` has no "update" action. To change a registered value (port, path, service), use `remove` then `add`; the cluster itself is never touched.

`check` says	Usually means	Fix
`DRIFT no registry file`	host never set up	`init` (root), then `import --tag`
`DRIFT malformed line N: …`	someone edited the file by hand	restore `/etc/edb/toolkit/epastab.prev` (the copy before the last write) or delete that line, as enterprisedb
`DRIFT as15: data directory … does not exist`	cluster decommissioned or moved	gone: `remove --name as15`; moved: `remove`, then `add` with the new `--data`
`DRIFT as17: PG_VERSION is 18, registry says 17`	upgraded in place, or wrong path registered	the name carries the version: `remove`, then `add --name as18 …`
`DRIFT as17: configured port is 5450, registry says 5444`	someone changed `port` (check `postgresql.auto.conf` too: `ALTER SYSTEM` wins)	intended: `remove` + `add --port 5450`; not intended: fix the config, then restart
`DRIFT as17: running on port 5450, registry says 5444`	same, and the server already runs with it	same choice; clients using `service=as17` currently connect to the wrong port
`DRIFT as17: service edb-as-17 is not installed`	package removed or unit renamed	correct unit: `remove` + `add --service <unit>`; pg_ctl-managed: `--service -`

`check` says	Usually means	Fix
`DRIFT as17: service edb-as-17 runs PGDATA=/x, not /u001/data/as17`	the systemd drop-in points elsewhere	root fixes `/etc/systemd/system/edb-as-17.service.d/override.conf`, then `systemctl daemon-reload`; or the registry path is wrong
`DRIFT running cluster not registered: version 17, /u001/data/x, port 5447`	a cluster started by hand, or a forgotten restore test	wanted: `add --name as17_<suffix> …`; not wanted: find its owner, then stop it
`NOTE … not readable as <user>`	you ran it as the wrong user	not drift: re-run as enterprisedb or root

After each fix, run `check` again until it ends with `RESULT registry matches this host` (exit code 0). The `epas` / `epasenv` switch also warns about port drift when you switch, so you will notice it during daily work.

> **Pro tips**
>
> - Run `check` from cron or your monitoring and alert on exit code 2: `/opt/edb_toolkit/tools/epastab.sh check >/tmp/epastab.check || mail -s "epastab drift $(hostname -s)" dba@… </tmp/epastab.check`.
> - Make `check` the last step of every change: patch, upgrade, standby rebuild, port change. A change is not done until the registry agrees.
> - Most drift comes from "quick fixes" done outside the toolkit. When you find one, fix the process (a runbook, a module) as well as the registry.

6. Update the toolkit later, and roll back

An update is the same install as step 1 with a newer tag; a rollback swaps `/opt/edb_toolkit.prev` back in. Neither touches your clusters, the registry, the TDE pass files, `conf/site.env` or anyone's `.bashrc`.

6.1 Make the change (on your laptop)

```
cd edb_toolkit
# … edit …
make lint && make test              # same checks CI runs on every push
git commit -am "epastab: warn when pg_service.local.conf is unreadable"   #
say what changed, not "1"
git tag -a v1.3.2 -m "v1.3.2"
git push && git push --tags
```

6.2 Roll it out to a server

```
cat /opt/edb_toolkit/VERSION            # what runs now (note it down)

# Road A: Git
cd ~/edb_toolkit && git fetch --tags && git checkout v1.3.2
sudo tools/install.sh

# Road B: bundle (make bundle on the laptop, copy, sha256sum -c, untar, as in
step 1)
sudo /tmp/edb_toolkit/tools/install.sh
```

The confirmation line names both versions, e.g. `install edb_toolkit v1.3.2 (…) into /opt/edb_toolkit (current: v1.3.1 (…); it is kept as /opt/edb_toolkit.prev)`, and you will see `Kept host config: conf/site.env`.

Then a 30-second smoke test, as enterprisedb:

```
cat /opt/edb_toolkit/VERSION
/opt/edb_toolkit/tools/epastab.sh check     # RESULT registry matches this
host
. epasenv -l                                 # every cluster listed
epas as17 && pginfo | grep Detection        # Detection : library …
(/opt/edb_toolkit)
```

If the release changed `tools/pg_bashrc.txt`, the DBA `.bashrc` is **not** updated for you. Compare and copy it deliberately:

```
diff ~/.bashrc /opt/edb_toolkit/tools/pg_bashrc.txt
cp -p ~/.bashrc ~/.bashrc.orig.$(date +%F) && cp
/opt/edb_toolkit/tools/pg_bashrc.txt ~/.bashrc
```

6.3 Roll back

Fastest, one step back (the command `install.sh` printed at the end):

```
sudo mv /opt/edb_toolkit /opt/edb_toolkit.bad && sudo mv
/opt/edb_toolkit.prev /opt/edb_toolkit
cat /opt/edb_toolkit/VERSION                # the old version again
```

Cleanest, to any version: reinstall the older tag (`git checkout v1.3.1 && sudo tools/install.sh` , or its bundle). `/usr/local/bin/epasenv` points at the `/opt/edb_toolkit` path, so it follows either way.

Know the limits of `.prev` :

- It holds **one** previous version. Two installs in a row and the version before last is gone; for anything older, reinstall the tag.
- It covers the toolkit only. A registry change is undone separately: `/etc/edb/toolkit/epastab.prev` is the registry before its last write.
- Delete `/opt/edb_toolkit.bad` once you know why the new version failed.

> **Pro tips**
>
> - Roll out by environment: DEV → SIT → PAT → PROD, with a few days in between. The tags make it obvious which server runs what.
> - Keep a one-line log per rollout (server, from, to, date). `cat /opt/edb_toolkit/VERSION` on each host, or an Ansible ad-hoc run, rebuilds it in minutes.
> - Never hot-fix a script in `/opt` during an incident. Use the rollback, then fix it in Git.

7. Gotchas I found in the repo while writing this

One of these will bite on the first real server: `pgstart` / `pgstop` via systemd will ask enterprisedb for a sudo password. The rest are small.

1. **sudoers does not match what pgstart runs.** `pg_bashrc.txt` calls `sudo systemctl start edb-as-17.service` ; `conf/sudoers.d/epas-automation` allows `/bin/systemctl start edb-as-17` , without `.service` . sudo compares arguments

literally, so the rule does not apply. Fix one side: add the `.service` forms to the sudoers file, or drop `.service` in `__epas_ctl`. (The build script's own hint, `sudo systemctl start edb-as-17`, does match.)

2. **sudoers has no EPAS 16 lines**, only 15 and 17. Its header also says `config/sudoers.d/` where the repo has `conf/sudoers.d/`.
3. **`epastab.sh` is not on PATH.** Only `epasenv` gets a `/usr/local/bin` link. Either add a second link in `install.sh` or the `alias epastab=…` from step 3 to `pg_bashrc.txt`.
4. **`.prev` keeps one version.** Fine if you know it (step 6.3); worth one line in the README.
5. **The README has no "Install on a server" section** (still on the roadmap). Sections 1, 4 and 6 of this guide could be cut down into it.
6. **Already parked, still open:** `lib/logger.sh` makes `deploy.log` world-writable (anyone can forge `[AUDIT]` lines), and clusters built with the old script have the TDE passphrase in `postgresql.conf`.

The next roadmap step still stands: run the split build on one real non-prod server and bring back the `epastab.sh check` output. That run will also confirm the example output in this guide.